

21 - 实际项目案例

阅读时间：约 3 小时

前置知识：06 - 安装与上手、18 - WebUI 实战、20 - Gateway 与生产部署

学习目标：掌握 3 个真实场景的 Nanobot 部署方案

目录

- 21.1 案例一：个人知识管理助手
 - 21.2 案例二：自动化运维 Bot
 - 21.3 案例三：多平台客服系统
-

21.1 案例一：个人知识管理助手

场景描述

构建一个能够自动整理笔记、生成摘要、跨会话记忆的个人知识助手。

技术选型

- **Provider**: DeepSeek (性价比高)
- **Channel**: WebSocket (WebUI)
- 核心功能: Dream 记忆系统、WebUI、Goal 系统

配置示例

```
{
  "providers": {
    "deepseek": {
      "apiKey": "${DEEPSEEK_API_KEY}"
    }
  },
  "modelPresets": {
    "primary": {
      "provider": "deepseek",
      "model": "deepseek-chat",
      "maxTokens": 8192,
      "contextWindowTokens": 65536
    }
  },
  "agents": {
    "defaults": {
      "modelPreset": "primary"
    }
  },
  "channels": {
    "websocket": {
      "enabled": true,
      "host": "127.0.0.1",
      "port": 8765
    }
  }
}
```

AGENTS.md 配置

个人知识管理助手

你是一个专业的知识管理助手，擅长：

核心能力

1. **** 笔记整理 ****: 将零散笔记整理为结构化文档
2. **** 摘要生成 ****: 生成长文摘要和关键点提取
3. **** 知识关联 ****: 发现不同笔记之间的关联
4. **** 记忆管理 ****: 自动记录重要知识点到 `MEMORY.md`

行为准则

- 使用 ``read_file`` 查阅笔记
- 使用 ``write_file`` 生成整理后的文档
- 使用 ``save_memory`` 记录关键知识点
- 保持文档结构清晰

输出格式

使用 **Markdown** 格式，包含：

- 标题层级
- 代码块（带语法高亮）
- 表格（对比数据）
- 链接（关联文档）

部署步骤

1. 创建工作目录

```
mkdir ~/knowledge-assistant && cd ~/knowledge-assistant
```

2. 创建配置文件

```
cat > config.json << 'EOF'
{配置内容}
EOF
```

3. 创建 AGENTS.md

```
cat > AGENTS.md << 'EOF'
{AGENTS.md 内容}
EOF
```

4. 启动 Gateway

```
nanobot gateway
```

5. 访问 WebUI

```
# 浏览器打开 http://127.0.0.1:8765
```

Dream 系统效果

运行一段时间后：

查看自动生成的记忆

```
cat memory/MEMORY.md
```

查看 Dream 处理进度

```
cat memory/.dream_cursor
```

```
# 查看会话历史
tail -20 memory/history.jsonl
```

21.2 案例二：自动化运维 Bot

场景描述

构建一个定时检查系统状态、发送告警通知的运维 Bot。

技术选型

- **Provider:** OpenAI（稳定性要求高）
- **Channel:** Telegram + Email
- 核心功能: Cron 定时任务、MCP 工具、多渠道通知

配置示例

```
{
  "providers": {
    "openai": {
      "apiKey": "${OPENAI_API_KEY}"
    }
  },
  "modelPresets": {
    "primary": {
      "provider": "openai",
      "model": "gpt-4o",
      "maxTokens": 4096
    }
  },
  "agents": {
    "defaults": {
      "modelPreset": "primary"
    }
  },
  "channels": {
    "telegram": {
      "enabled": true,
      "token": "${TELEGRAM_TOKEN}"
    },
    "email": {
      "enabled": true,
      "imapHost": "imap.gmail.com",
      "smtpHost": "smtp.gmail.com",
      "email": "bot@example.com",
      "imapPassword": "${EMAIL_PASSWORD}",
      "smtpPassword": "${EMAIL_PASSWORD}"
    }
  },
  "tools": {
    "mcpServers": {
      "system-monitor": {
        "command": "python",
        "args": ["monitor_server.py"]
      }
    }
  }
}
```

```

    }
  },
  "gateway": {
    "heartbeat": {
      "enabled": true,
      "interval": 1800
    }
  }
}
}

```

Cron 定时任务

通过 `/goal` 或配置文件设置定时任务：

每天早上 9 点：检查服务器状态

每 6 小时：检查磁盘空间

收到告警时：发送邮件通知

AGENTS.md 配置

运维助手

你是一个 7x24 小时的运维助手。

职责

1. ** 系统监控 **：定期检查服务器状态
2. ** 告警通知 **：发现问题时发送 Telegram 和 Email
3. ** 日志分析 **：分析错误日志，定位问题
4. ** 自动修复 **：执行预定义的修复脚本

工具使用

- 使用 MCP ``system-monitor`` 检查服务器
- 使用 ``exec`` 执行诊断命令
- 使用 ``read_file`` 查看日志
- 通过 Telegram 和 Email 发送通知

告警规则

- CPU > 90% 持续 5 分钟 → 告警
- 磁盘 > 85% → 告警
- 服务宕机 → 立即告警并尝试重启

部署步骤

1. 创建工作目录

```
mkdir ~/ops-bot && cd ~/ops-bot
```

2. 创建配置

```
cat > config.json << 'EOF'
{配置内容}
EOF
```

3. 创建 MCP Server

```
cat > monitor_server.py << 'EOF'
# MCP 服务器代码（检查系统状态）
EOF
```

4. 启动

nanobot gateway

5. 验证

nanobot status

21.3 案例三：多平台客服系统

场景描述

构建一个同时支持 Telegram、飞书、微信的客服系统，使用 Goal 系统维持服务目标。

技术选型

- **Provider:** Anthropic Claude（对话质量高）
- **Channel:** Telegram + Feishu + Weixin
- 核心功能：多通道、Goal 系统、Skills 知识库

配置示例

```
{
  "providers": {
    "anthropic": {
      "apiKey": "${ANTHROPIC_API_KEY}"
    }
  },
  "modelPresets": {
    "primary": {
      "provider": "anthropic",
      "model": "claude-sonnet-4-20250514",
      "maxTokens": 8192,
      "contextWindowTokens": 65536
    }
  },
  "agents": {
    "defaults": {
      "modelPreset": "primary"
    }
  },
  "channels": {
    "telegram": {
      "enabled": true,
      "token": "${TELEGRAM_TOKEN}"
    },
    "feishu": {
      "enabled": true,
      "appId": "${FEISHU_APP_ID}",
      "appSecret": "${FEISHU_APP_SECRET}"
    },
    "weixin": {
      "enabled": true,
      "token": "${WEIXIN_TOKEN}"
    }
  }
}
```

Skills 知识库

创建客服知识库：

```
workspace/  
└─ skills/  
    └─ customer-service/  
        └─ SKILL.md
```

SKILL.md 内容：

Customer Service Skill

激活条件

当用户咨询产品相关问题时激活。

知识库

产品 A

- 价格：¥99/月
- 功能：自动备份、数据分析
- 常见问题：...

产品 B

- 价格：¥199/月
- 功能：团队协作、API 访问
- 常见问题：...

回答规范

1. 先确认用户问题属于哪个产品
2. 从知识库查找答案
3. 如果找不到，使用 `web_search` 搜索
4. 保持语气友好、专业

AGENTS.md 配置

客服助手

你是一个专业的多平台客服助手。

职责

1. **** 问题解答 ****：回答用户关于产品的咨询
2. **** 问题排查 ****：帮助用户解决使用问题
3. **** 工单创建 ****：复杂问题创建工单并转人工
4. **** 满意度调查 ****：服务结束后询问满意度

使用 Goal 系统

对于复杂问题，使用 ``/goal`` 命令：

- 跟踪问题解决进度
- 确保不遗漏任何步骤
- 跨会话维持上下文

服务流程

1. 问候用户

2. 确认问题
3. 查找解决方案
4. 提供解答
5. 确认是否解决
6. 满意度调查

部署步骤

1. 创建工作目录

```
mkdir ~/customer-service && cd ~/customer-service
```

2. 创建配置

```
cat > config.json << 'EOF'
{配置内容}
EOF
```

3. 创建 Skills

```
mkdir -p skills/customer-service
cat > skills/customer-service/SKILL.md << 'EOF'
{SKILL.md 内容}
EOF
```

4. 启动

```
nanobot gateway
```

5. 测试多平台

分别从 Telegram、飞书、微信发送消息

Goal 系统实战

处理复杂问题:

用户: 我无法登录系统

Agent: /goal 帮助用户解决登录问题

[Goal Active]

目标: 帮助用户解决登录问题

[第1轮] 确认用户账号状态...

[第2轮] 检查密码是否正确...

[第3轮] 发现账号被锁定...

[第4轮] 解锁账号...

[第5轮] 指导用户重置密码...

[Goal Completed]

问题已解决, 账号已解锁。

下一章: 22 - 测试与可观测性